Lab 6: Binary Search Tree (BST)

Data Structures and Algorithms

523c0012 - Huynh Nhat Huy

April 1, 2026

Introduction

This report details the completion of the exercises for Lab 6 on Binary Search Trees (BST)[cite: 5, 6]. The implementations are written in Java and fulfill the requirements for the Data Structures and Algorithms course at Ton Duc Thang University[cite: 1, 3, 4].

Exercise 1: BST Class Implementation

Based on the provided sections, a complete BST class was implemented along with a main function for testing[cite: 348, 349].

```java
public class Node {
    Integer key;
    Node left, right;

    public Node(Integer key) {
        this.key = key;
        this.left = this.right = null;
    }
}

public class BST {
    Node root;
    // Core insertion, search, and delete methods implemented
       here
}
```

Listing 1: Node and Basic BST Structure

Exercise 2: Create Tree from String

The `createTree(String strKey)` function parses a string of integers separated by spaces and constructs a BST[cite: 352, 353].

```java
public void createTree(String strKey) {
    String[] keys = strKey.split(" ");
```

```
    for (String k : keys) {
        root = insert(root, Integer.parseInt(k));
    }
}
```

Listing 2: createTree Method

Exercise 3: Descending Order Traversal

To print the values in descending order, a Reverse In-order traversal (Right-Node-Left) was implemented[cite: 355, 356].

```
private void printDescending(Node x) {
    if (x != null) {
        printDescending(x.right);
        System.out.print(x.key + " ");
        printDescending(x.left);
    }
}

public void printDescending() {
    printDescending(root);
}
```

Listing 3: Descending Order Method

Exercise 4: Contains Key

The `contains` function returns true if a given key exists in the tree, and false otherwise[cite: 358, 359].

```
public boolean contains(Integer key) {
    return search(root, key) != null;
}
```

Listing 4: Contains Method

Exercise 5: Delete Maximum Key

The `deleteMax()` function traverses the right links to locate and remove the node containing the maximum key[cite: 368].

```
private Node deleteMax(Node x) {
    if (x.right == null) return x.left;
    x.right = deleteMax(x.right);
    return x;
}

public void deleteMax() {
    root = deleteMax(root);
```

```
}
```

Listing 5: deleteMax Method

Exercise 6: Delete Node using Predecessor

This function deletes a node by substituting it with its predecessor (the maximum value in its left subtree) instead of the successor[cite: 371, 372].

```
public void delete_pre(Integer key) {
    root = delete_pre(root, key);
}

private Node delete_pre(Node x, Integer key) {
    if (x == null) return null;
    int cmp = key.compareTo(x.key);
    if (cmp < 0) x.left = delete_pre(x.left, key);
    else if (cmp > 0) x.right = delete_pre(x.right, key);
    else {
        if (x.right == null) return x.left;
        if (x.left == null) return x.right;

        Node t = x;
        x = max(t.left);
        x.left = deleteMax(t.left);
        x.right = t.right;
    }
    return x;
}
```

Listing 6: delete_pre Method

Exercise 7: Height of BST

The `height()` function recursively calculates the depth of the tree structure[cite: 377, 378].

```
private int height(Node x) {
    if (x == null) return -1;
    return 1 + Math.max(height(x.left), height(x.right));
}

public int height() {
    return height(root);
}
```

Listing 7: Height Method

Exercise 8: Sum of All Keys

An auxiliary function calls the recursive `sum(Node x)` function to compute the total sum of all keys in the tree[cite: 382, 383].

```
public Integer sum(Node x) {
    if (x == null) return 0;
    return x.key + sum(x.left) + sum(x.right);
}

public Integer sum() {
    return sum(root);
}
```

Listing 8: Sum Method

Exercise 9: Sum of Even Keys

The `sumEven()` function calculates the total sum of strictly even keys[cite: 400, 401].

```
private Integer sumEven(Node x) {
    if (x == null) return 0;
    int val = (x.key % 2 == 0) ? x.key : 0;
    return val + sumEven(x.left) + sumEven(x.right);
}

public Integer sumEven() {
    return sumEven(root);
}
```

Listing 9: sumEven Method

Exercise 10: Count Leaves

The `countLeaves()` function traverses the tree and counts nodes that have no children[cite: 404, 405].

```
private int countLeaves(Node x) {
    if (x == null) return 0;
    if (x.left == null && x.right == null) return 1;
    return countLeaves(x.left) + countLeaves(x.right);
}

public int countLeaves() {
    return countLeaves(root);
}
```

Listing 10: countLeaves Method

Exercise 11: Sum Even Keys at Leaves

This implementation finds leaf nodes specifically and adds their values to the sum only if they are even[cite: 408, 409].

```
private int sumEvenKeysAtLeaves(Node x) {
    if (x == null) return 0;
    if (x.left == null && x.right == null) {
        return (x.key % 2 == 0) ? x.key : 0;
    }
    return sumEvenKeysAtLeaves(x.left) + sumEvenKeysAtLeaves(x.
       right);
}

public int sumEvenKeysAtLeaves() {
    return sumEvenKeysAtLeaves(root);
}
```

Listing 11: sumEvenKeysAtLeaves Method

Exercise 12: Breadth-First Search (BFS)

The `bfs()` function utilizes a Queue to print the keys via level-order traversal[cite: 413, 414].

```
import java.util.LinkedList;
import java.util.Queue;

public void bfs() {
    if (root == null) return;
    Queue<Node> queue = new LinkedList<>();
    queue.add(root);

    while (!queue.isEmpty()) {
        Node current = queue.poll();
        System.out.print(current.key + " ");

        if (current.left != null) queue.add(current.left);
        if (current.right != null) queue.add(current.right);
    }
    System.out.println();
}
```

Listing 12: BFS Method